ARCHITECTURE ADR SYNTHESIS — VERSION 1

Phase: Final ADR Synthesis **Status:** COMPLETE **Date:** 2026-06-04 **Source Corpus:** 120 ADR candidates, 1 ACD, 1 Driver Registry, 1 Root Constraint Registry, 1 Quality Audit, 1 Readiness Assessment **Compression Ratio:** 120 candidates → 20 Architecture ADRs **Rejected / Derived:** 100 ADRs (see Section 5)

SECTION 1 — EXECUTIVE SUMMARY

Compression Logic

The 120-ADR candidate corpus contains four distinct layers of decision:

1. **Architecture decisions** — choices that change the fundamental shape of the system. These survive technology changes and remain true for 3–5 years. **Promoted.**
2. **Constraint expressions** — rules that flow directly from an architecture decision. They express the same architecture choice at a finer granularity. **Merged into parent ADR.**
3. **Validation requirements** — tests that prove an architecture decision is correctly implemented. Not decisions themselves. **Moved to Rejected/Derived.**
4. **Operational procedures** — how to run the system once built. Governance, runbooks, alert processes. Not architecture. **Moved to Rejected/Derived.**

The corpus had a structural problem: many ADRs in the Risk Control, Human Oversight, Validation, and Infrastructure categories were restatements of the same 5–6 underlying architecture decisions expressed at different levels of specificity. This synthesis resolves that by extracting the single architecture-level decision and retiring the duplicates.

Coverage Summary

Domain	ADR Count
System Scope & SEBI Regulatory Boundary	1
Local-First Embedded Storage Strategy	1
Market Data Storage Format	1
Market Data Reliability & Multi-Source Verification	1

Domain	ADR Count
Transaction Integrity & Crash Recovery	1
Walk-Forward Validation as Deployment Gate	1
Statistical Deployment Threshold	1
Drift Detection & Regime-Aware Execution	1
Execution Isolation & Determinism	1
Human Approval Gate Architecture	1
AI Domain Boundary	1
AI-to-Execution Physical Block	1
Risk Controls Architecture	1
Production Binary & Config Integrity	1
Data Ingestion Integrity	1
Backtesting Survivorship Integrity	1
API Rate Compliance & Retry Architecture	1
Secrets & Authentication Architecture	1
Scalability Transition Trigger	1
Concurrency Boundary	1
TOTAL	**20**

SECTION 2 — ARCHITECTURE ADR CORPUS

ARCH-001 — System Scope: SEBI-Regulated Deterministic Swing Trading System

ADR_ID: ARCH-001 **TITLE:** System Is a SEBI-Regulated, Human-Gated, Deterministic Execution System — Not an Autonomous Trading Bot **STATUS:** APPROVED

ARCHITECTURE_DOMAIN: System Scope

DECISION: This system is a SEBI-regulated quantitative swing trading platform operating on Indian equity markets under NSE/BSE exchange rules. The fundamental architecture boundary is deterministic execution with mandatory human approval gates. The system must never be designed, extended, or operated as a fully autonomous trading bot. This is not a soft policy preference — it is a hard system scope constraint enforced by SEBI regulatory requirements and validated as the highest-confidence finding (95/100) in the research corpus.

RATIONALE: SEBI regulations mandate static IP addressing, OAuth authentication, and 10 orders/second rate limiting. Direct AI execution of orders is rated 10/100 confidence safety in the corpus audit. The corpus's single highest-confidence claim is that autonomous AI execution within SEBI/Zerodha constraints will cause catastrophic failure. Any architecture that creates a path toward autonomous execution — even conditionally or experimentally — violates this foundational scope.

SOURCE_CONSTRAINTS: ACD-REG-001, ACD-REG-002, ACD-REG-003, ACD-HUM-001, ACD-AIB-001 **SOURCE_ADRS:** ADR-008, ADR-071, ADR-074, ADR-076, ADR-086, ADR-096

ALTERNATIVES_CONSIDERED:

- Fully autonomous AI execution: Rejected. SEBI non-compliant. 10/100 safety score. Direct Knight Capital failure analog.
- Semi-autonomous with override: Rejected. Creates a pathway that degrades to full autonomy under operational pressure.
- Human-gated with AI advisory: Selected. Compliant, auditable, reversible.

TRADEOFFS: Human gates introduce latency between signal and execution. This is acceptable for a swing trading strategy but would be incompatible with high-frequency execution. The system's scope is explicitly swing trading, which makes human-gated latency a non-issue.

VALIDATION_REQUIREMENTS:

- SEBI compliance documentation reviewed and confirmed against current regulatory rules
- Broker sandbox test confirming static IP + OAuth + rate limit parameters
- Paper trading run demonstrating human gate does not create execution failure under normal swing trading cadence

RISKS:

- Regulatory interpretation change: low probability, high impact. Mitigation: monitor SEBI circulars.

- Insider bypass of human gate: documented failure mode. Mitigation addressed by ARCH-010.

FUTURE_UPGRADE_TRIGGER: SEBI regulatory change permitting algorithmic execution without human approval gates, or system scope change from swing to intraday/HFT (requires complete architecture re-evaluation).

ARCHITECTURE_IMPACT: Foundational. All other architecture decisions inherit from this scope. Removing this decision collapses ARCH-010, ARCH-011, ARCH-012.

ARCH-002 — Local-First Embedded Storage Architecture

ADR_ID: ARCH-002 **TITLE:** All Analytical and Transactional Data Processing Must Use Local Embedded Storage — No Distributed Database **STATUS:** APPROVED
ARCHITECTURE_DOMAIN: Local-First Strategy / Storage Strategy

DECISION: The system must use a local embedded storage architecture for all data processing. Distributed databases, cloud-native databases, and vector databases are architecturally prohibited for the core data pathway. Storage must be co-located with the compute process that reads it to eliminate network dependency during live execution. The data processing engine and the transactional record store must each be embedded within the host process — they must not be separate services.

RATIONALE: 95% of trading data is structured time-series OHLCV. Vector databases are workload-mismatched. Network round-trips to distributed stores introduce non-deterministic latency during execution decisions. The corpus designates this as 90/100 confidence with a hard MUST constraint. DuckDB and SQLite are named in the corpus as the specific implementations satisfying this constraint, but the architecture decision is the local embedded requirement, not the specific technologies. Any future technologies that satisfy local-embedded, zero-network-hop, structured-time-series OHLCV query requirements would also satisfy this ADR.

SOURCE_CONSTRAINTS: ACD-DAT-005, ACD-DAT-006 **SOURCE_ADRS:** ADR-001, ADR-002, ADR-077

ALTERNATIVES_CONSIDERED:

- Remote/cloud database: Rejected. Network dependency during execution. Non-deterministic latency.
- Vector database: Rejected. Workload mismatch for structured OHLCV time-series.

- Row-oriented embedded database for analytics: Rejected. Time-series aggregation bottleneck under multi-year scan loads (§7 Finding 24).
- Columnar analytical engine + row-based transactional store (co-embedded): Selected.

TRADEOFFS: Local storage bounds horizontal scalability. Analytical query performance is bound by local RAM and disk. Concurrent write-read performance at large data scales (50GB+) is an unresolved benchmark gap. These tradeoffs are acceptable given the system's local-first, single-operator design scope at current stage.

VALIDATION_REQUIREMENTS:

- Benchmark: analytical store performance on multi-year 1-minute OHLCV at realistic scale
- Benchmark: concurrent read performance under live write conditions
- Chaos test: process crash during write — verify recovery to consistent state
- Load test: memory ceiling under full historical scan

RISKS:

- Concurrency bottleneck at 50GB+ scale: unresolved (GAP-08). This is a known open risk requiring empirical validation before architecture is finalized.
- Single-host failure: mitigated by ARCH-005 (async backup strategy).

FUTURE_UPGRADE_TRIGGER: Data volume exceeds single-host capacity threshold established by benchmark validation. Operator count grows beyond single-host concurrency limits. Regulatory requirement for off-host audit trail.

ARCHITECTURE_IMPACT: HIGH. Determines data flow topology, process hosting model, and scalability ceiling. Dependency for ARCH-003, ARCH-005, ARCH-015.

ARCH-003 — Market Data Storage Format

ADR_ID: ARCH-003 **TITLE:** Market Data Must Be Stored in Time-Partitioned Columnar Format — Unstructured Formats Prohibited **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Market Data Storage Format / Data Governance

DECISION: All market data — historical OHLCV, intraday feeds, reference data — must be stored in a partitioned columnar format organized by time dimension (Year/Month at minimum). Unstructured, flat, or document-oriented formats (JSON lakes, CSV flat files, raw text) are prohibited as the primary storage format for any market data that will be queried analytically.

The partition structure must align with the analytical query patterns of the downstream engine (time-range scans).

RATIONALE: The corpus mandates Hive-partitioned Parquet as a hard MUST (§19). The architecture-level principle is time-partitioned columnar storage, not a specific file format. Any future format satisfying columnar compression, time-range partition pruning, and embedded engine scan compatibility satisfies this ADR. The prohibition on unstructured formats is absolute — it prevents query performance degradation, memory exhaustion on full-scan reads, and schema drift.

SOURCE_CONSTRAINTS: ACD-DAT-005 **SOURCE_ADRS:** ADR-003, ADR-078

ALTERNATIVES_CONSIDERED:

- JSON data lake: Rejected. Full scan memory exhaustion. No partition pruning.
- Flat CSV per symbol: Rejected. No partition strategy. Schema inconsistency risk.
- Time-partitioned columnar format: Selected.

TRADEOFFS: Write path is more complex than flat file append. Partition management requires schema discipline. Memory overhead during scan plan building on large partition trees is a known risk.

VALIDATION_REQUIREMENTS:

- Verify partition structure aligns with query patterns before schema is finalized
- Inject synthetic anomalies into ingestion pipeline; verify schema enforcement
- Memory profiling of scan plan under maximum historical depth

RISKS:

- Partition explosion from over-granular time keys: mitigated by Year/Month floor
- Schema drift across ingestion sources: mitigated by ARCH-006 (multi-source validation)

FUTURE_UPGRADE_TRIGGER: Analytical engine changes that require a different on-disk format. Regulatory requirement for a specific archival format. Data volume requiring hierarchical time keys below Month granularity.

ARCHITECTURE_IMPACT: MEDIUM. Determines data pipeline schema contract and ingestion design. Dependency for ARCH-002, ARCH-006.

ARCH-004 — Market Data Reliability: Multi-Source Cross-Verification

ADR_ID: ARCH-004 **TITLE:** All Execution-Bound Signal Data Must Be Cross-Verified Across a Minimum of Two Independent Sources **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Market Data Reliability / Data Governance

DECISION: No single external data provider may serve as the sole authoritative source for any data type that feeds execution-bound signal logic. The data pipeline must have a mandatory cross-verification layer that compares OHLCV metrics from at least two structurally independent providers before any data is committed to the validated data store. Structurally independent means: different data collection infrastructure, different upstream exchange connections, not white-labeled from the same primary source. Any provider that fails independent verification must trigger quarantine, not silent acceptance.

RATIONALE: Zerodha historical data is structurally incomplete (§7 Finding 2). Provider-specific dropped candles, silent API changes, and split adjustment errors are documented failure modes (ADR-004, ADR-011). Single-source pipelines cannot detect provider-specific corruption. A cross-verification architecture is the minimal detection mechanism.

SOURCE_CONSTRAINTS: ACD-DAT-001, ACD-DAT-002, ACD-DAT-008 **SOURCE_ADRS:** ADR-004, ADR-005, ADR-011, ADR-012, ADR-041

ALTERNATIVES_CONSIDERED:

- Single authoritative provider: Rejected. Structural incompleteness documented for primary provider.
- Single provider with anomaly detection: Rejected. Cannot detect provider-specific systematic errors (all candles wrong, not missing).
- Two independent providers with cross-verification: Selected.

TRADEOFFS: Doubles data acquisition complexity. Adds latency to data ingestion pipeline. Creates dependency on two provider API contracts instead of one. All tradeoffs are acceptable given the capital risk of uncorrected data corruption.

VALIDATION_REQUIREMENTS:

- Inject synthetic dropped candles; verify quarantine triggers
- Inject synthetic price anomalies; verify cross-verification failure path activates
- Verify independence of two providers (different upstream infrastructure)
- Confirm corporate action adjustment methodology per provider is documented

RISKS:

- Both providers produce identical corruption simultaneously (shared upstream): low probability, accepted
- Provider 2 downtime leaving single-source gap: requires defined handling policy (accept with flag, halt, or use last-known-good)

FUTURE_UPGRADE_TRIGGER: Regulatory change requiring specific data sources. Provider infrastructure convergence reducing independence. Addition of new asset classes requiring provider-specific coverage.

ARCHITECTURE_IMPACT: HIGH. Determines data pipeline topology, provider contract dependencies, and ingestion validation design.

ARCH-005 — Transaction Integrity and Crash Recovery Architecture

ADR_ID: ARCH-005 **TITLE:** Transaction Records Must Survive Hard System Failure and Be Recoverable to Consistent State Without Manual Intervention **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Transaction Integrity / Reliability

DECISION: The transactional record store must use a write-ahead mechanism that guarantees committed state survives a hard process crash or power loss. Recovery to consistent state must be fully automated — no manual reconciliation step is permitted in the recovery path. An asynchronous off-host backup of the transaction log must be continuously streaming such that data loss on unclean shutdown is bounded to a defined maximum interval. WAL files must be bounded in size — unbounded WAL growth on backup stream hang is prohibited. NSE trade cancellation events must be handled as explicit, atomic rollbacks, not silent discards.

RATIONALE: SQLite WAL is identified as a systemic failure node (§21). Litestream replication assumptions are unvalidated under hard power-off (§9 A2, A18). Unbounded WAL growth on S3 upload hang is a documented failure mode. Trade void events from NSE clearinghouse must produce clean rollbacks, not corrupted position state. The architecture must solve these as a compound problem — WAL integrity + bounded growth + off-host backup + explicit void handling.

SOURCE_CONSTRAINTS: ACD-DAT-006, ACD-DAT-009, ACD-OPS-003 **SOURCE_ADRS:** ADR-015, ADR-019, ADR-020, ADR-043, ADR-079, ADR-080, ADR-114

ALTERNATIVES_CONSIDERED:

- In-process memory-only state: Rejected. No crash recovery.
- Remote database with transactional guarantees: Rejected. Network dependency during execution (contradicts ARCH-002).
- Local write-ahead + async off-host replication: Selected.

TRADEOFFS: Async replication means the off-host backup trails the live state by the replication interval. Maximum data loss is bounded, not zero. This is acceptable for swing trading but must be validated against the actual recovery SLA requirement. WAL management adds operational complexity.

VALIDATION_REQUIREMENTS:

- Force hard power-off during active write; verify recovery to consistent state
- Disconnect off-host backup stream during write; verify WAL growth stays bounded
- Inject NSE trade void event; verify atomic rollback without manual intervention
- Measure maximum data loss interval under realistic write throughput

RISKS:

- Replication validation not yet completed (ADR-015, ADR-043 remain PROVISIONAL): this is a blocker for architecture finalization
- Network connectivity to off-host backup: requires validated SLA

FUTURE_UPGRADE_TRIGGER: Recovery time objective changes that require synchronous off-host replication. Capital buffer requirement for trade cancellations changes with new SEBI clearing rules.

ARCHITECTURE_IMPACT: HIGH. Determines transactional storage design, backup topology, and position reconciliation logic.

ARCH-006 — Data Ingestion Integrity: Anomaly Detection and Completeness Verification

ADR_ID: ARCH-006 **TITLE:** Data Ingestion Must Include a Mandatory Validation Layer That Detects Anomalies, Completeness Failures, and Pricing Errors Before Data Reaches Signal Logic **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Data Ingestion Integrity

DECISION: The data pipeline must contain a non-optional validation stage between raw ingestion and validated storage. This stage must: (1) verify intraday feed completeness (no dropped candles against expected session length); (2) detect pricing anomalies via cross-source

comparison; (3) halt downstream processing on bad pricing data — not log and continue; (4) be proven against synthetic anomaly injection before it is trusted. Validation failures must route to a quarantine path, not a silent passthrough. The validation pipeline itself is an architectural component, not a testing concern.

RATIONALE: Bad pricing data triggers infinite downstream loops (§10 Mode 24). Dropped candles produce false indicators without detection. Cross-source verification requires a structured validation stage to execute. The validation pipeline must be a first-class architectural component with its own tested failure paths.

SOURCE_CONSTRAINTS: ACD-DAT-002, ACD-DAT-004, ACD-DAT-007 **SOURCE_ADRS:** ADR-012, ADR-016, ADR-017, ADR-040, ADR-044, ADR-118

ALTERNATIVES_CONSIDERED:

- Validate at signal computation time: Rejected. Bad data already committed. Partial signals computed.
- Log-only anomaly detection: Rejected. Does not prevent bad data from reaching execution logic.
- Mandatory quarantine-on-failure validation stage: Selected.

TRADEOFFS: Adds latency to the data ingestion pipeline. Requires maintaining expected session length calendars (market holidays, half-sessions). Quarantine path requires an operator resolution workflow.

VALIDATION_REQUIREMENTS:

- Inject synthetic dropped candles; verify completeness check activates and quarantine fires
- Inject synthetic pricing spike; verify cross-source discrepancy detection activates
- Verify downstream halt fires on bad data (not just log entry)
- Prove validation pipeline with synthetic anomaly suite before live data trusted

RISKS:

- Validation false positive rate causing excessive quarantine events: requires threshold calibration
- Incorrect session length calendar on market holidays: explicitly tested

FUTURE_UPGRADE_TRIGGER: New data types added that require different completeness verification logic. Addition of new exchange or market with different session structure.

ARCHITECTURE_IMPACT: MEDIUM-HIGH. Determines data pipeline topology and the trust boundary of validated storage.

ARCH-007 — Walk-Forward Validation as the Mandatory Deployment Gate

ADR_ID: ARCH-007 **TITLE:** No Strategy May Be Deployed to Live Capital Without Walk-Forward Out-of-Sample Validation — Randomized Cross-Validation Is Architecturally Prohibited **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Validation Framework

DECISION: The strategy validation pipeline must enforce walk-forward cross-validation as the sole permitted deployment gate. Randomized, k-fold, or any methodology that samples training data without preserving strict temporal ordering is architecturally prohibited. The validation pipeline must be implemented such that data leakage from future windows into training windows is structurally impossible — not merely policy-prohibited. The validation component must expose a deployment gate interface that returns a binary pass/fail against both the walk-forward requirement and the statistical significance threshold (see ARCH-008).

RATIONALE: Randomized CV allows future data to leak into training windows, producing optimistically biased performance metrics that do not generalize to live markets. Walk-forward OOS validation is the only methodology that mimics the causal structure of live trading (train on past, test on future). This is a structural requirement on the validation pipeline architecture, not a user-settable parameter.

SOURCE_CONSTRAINTS: ACD-VAL-001 **SOURCE_ADRS:** ADR-007, ADR-027, ADR-046

ALTERNATIVES_CONSIDERED:

- K-fold cross-validation: Rejected. Data leakage. Non-causal sampling.
- Walk-forward with expanding window: Selected.
- Walk-forward with rolling window: Acceptable variant — window type must be explicitly configured and documented per strategy.

TRADEOFFS: Walk-forward validation requires substantially more historical data than k-fold (each fold needs sufficient history for signal warmup). Validation runtime is longer. Both tradeoffs are acceptable — cutting corners on validation methodology is an unacceptable capital risk.

VALIDATION_REQUIREMENTS:

- Test validation pipeline with known-leaky input; verify it is rejected

- Verify temporal ordering constraint is structural (code-enforced) not configurable
- Confirm minimum historical depth requirement per strategy type is documented

RISKS:

- Walk-forward OOS fidelity to future regime changes is unvalidated (ADR-046): this is an accepted open limitation, not a reason to weaken the gate
- Insufficient historical depth for some strategy types: requires minimum data requirement per strategy class

FUTURE_UPGRADE_TRIGGER: Research validates an alternative methodology with superior regime fidelity while maintaining temporal causality.

ARCHITECTURE_IMPACT: HIGH. Determines the architecture of the validation pipeline and what constitutes a valid deployment decision.

ARCH-008 — Statistical Significance Threshold for Strategy Deployment

ADR_ID: ARCH-008 **TITLE:** Strategy Deployment Requires a Statistical Significance Threshold More Conservative Than Standard 95% Confidence — t-stat ⩽ 2.0 Is Insufficient **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Validation Framework

DECISION: The deployment gate must enforce a minimum statistical significance threshold that accounts for multiple hypothesis testing bias (data mining). The threshold must be more conservative than a standard 95% confidence interval (t-statistic = 2.0). The deployment gate must implement either a Deflated Sharpe Ratio test or an equivalent multiple-testing-corrected measure. A strategy that passes only t-stat ⩾ 2.0 must be blocked from deployment. The specific threshold value (e.g., t-stat ⩾ 3.0) must be empirically calibrated against the strategy search space size and documented as a versioned configuration parameter.

RATIONALE: Data mining over large strategy search spaces dramatically inflates the probability of finding spurious alpha. Standard 95% confidence is insufficient when hundreds of parameter combinations have been tested. The corpus documents this as a validated failure mode. The deployment gate must encode this mathematical reality structurally.

SOURCE_CONSTRAINTS: ACD-VAL-002 **SOURCE_ADRS:** ADR-028, ADR-029

ALTERNATIVES_CONSIDERED:

- Standard 95% CI (t=2.0): Rejected. Insufficient under data mining conditions.
- No minimum threshold: Rejected. Permits noise-fitted strategies.

- Deflated Sharpe / multiple-testing-corrected threshold: Selected.

TRADEOFFS: More conservative threshold rejects some genuinely profitable strategies. This is the correct tradeoff — false negatives (missed alpha) are recoverable; false positives (deployed noise) cause capital loss.

VALIDATION_REQUIREMENTS:

- Verify deployment gate rejects strategies that pass t=2.0 but fail the higher threshold
- Calibrate threshold value against documented strategy search space size
- Document threshold version history alongside deployed strategy versions

RISKS:

- Threshold too conservative, rejecting all strategies: requires calibration evidence
- Threshold documented but not enforced structurally: must be code-enforced, not advisory

FUTURE_UPGRADE_TRIGGER: Change in strategy search methodology that alters the effective hypothesis space. Regulatory change requiring a specific statistical standard.

ARCHITECTURE_IMPACT: MEDIUM. Determines the deployment gate logic in the validation pipeline.

ARCH-009 — Drift Detection and Regime-Aware Execution

ADR_ID: ARCH-009 **TITLE:** The System Must Continuously Monitor for Distributional Shift Between Live Data and Training Data — Regime Change Must Trigger Automatic Execution Response **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Validation Framework / Risk Controls

DECISION: The system must include a continuous distribution monitoring component that tracks shift between live market data and the training distribution of deployed models and strategies. When distributional shift exceeds a calibrated threshold, the system must automatically reduce position sizing or halt execution — not merely generate an alert. The monitoring component must be architecturally integrated with the execution path, not an offline reporting tool. Autoencoder-based anomaly classifiers used in this layer must be audited to ensure they do not mask the structural shifts they are meant to detect.

RATIONALE: Market regimes change. Models validated in one regime degrade in another. AI grids cannot adapt across regime changes (§8 Finding 7). The corpus documents concept drift as

a systemic failure mode. A monitoring component that only alerts is insufficient — the response must be automated and architecturally integrated into execution scaling.

SOURCE_CONSTRAINTS: ACD-VAL-006 **SOURCE_ADRS:** ADR-034, ADR-048, ADR-058, ADR-063

ALTERNATIVES_CONSIDERED:

- Manual periodic review of model performance: Rejected. Cannot respond within live trading session.
- Alert-only monitoring: Rejected. Requires human to take action; too slow; subject to alert fatigue.
- Automated scale-down on threshold breach: Selected.

TRADEOFFS: Automated scale-down may trigger on statistical noise rather than genuine regime change. This requires threshold calibration and creates false-positive operational cost. Accepted — false positive (unnecessary scale-down) is preferable to false negative (full-size execution into wrong regime).

VALIDATION_REQUIREMENTS:

- Replay March 2020 crash data; verify drift detection triggers before strategy peak drawdown
- Audit anomaly classifier: verify it does not mask the structural shifts it monitors
- Calibrate PSI or equivalent threshold against historical regime change events

RISKS:

- Autoencoder masking genuine anomalies (ADR-063): known risk, requires explicit audit before live deployment
- Threshold calibration: empirical work required

FUTURE_UPGRADE_TRIGGER: Change in model architecture that requires different distributional monitoring methods. Addition of new strategy types with fundamentally different distributional properties.

ARCHITECTURE_IMPACT: MEDIUM-HIGH. Determines architecture of the monitoring layer and its integration with execution scaling logic.

ARCH-010 — Human Approval Gate Architecture

ADR_ID: ARCH-010 **TITLE:** The Execution Path Must Contain a Mandatory Human Approval Gate — No Order May Reach an Exchange Without a Human Decision Event in Its Causal Chain **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Human Approval Model / Execution Isolation

DECISION: The architecture must enforce a mandatory human approval gate in the critical path between signal generation and order submission. This gate must be a structural component — not a policy, not a configurable flag, not a feature toggle. The gate must require a positive human confirmation action for each order or order batch. The gate must not be bypassable by any automated component, AI component, or internal system process. Position limit overrides and risk parameter expansions must require multi-party authorization (minimum two human approvals). Volume growth beyond authorized parameters must be physically blocked, not merely policy-prohibited.

RATIONALE: SEBI regulatory mandate (ADR-071, 95/100 confidence). Knight Capital failure class: automated systems that remove human gates fail catastrophically and irreversibly. The corpus's highest-confidence finding is that deterministic human-gated execution is the non-negotiable execution architecture for this system and regulatory context.

SOURCE_CONSTRAINTS: ACD-HUM-001, ACD-HUM-003, ACD-HUM-004 **SOURCE_ADRS:** ADR-069, ADR-070, ADR-071, ADR-073, ADR-076, ADR-096

ALTERNATIVES_CONSIDERED:

- Fully autonomous execution: Rejected. SEBI non-compliant. 10/100 safety. Direct Knight Capital analog.
- Human override (automated with opt-out): Rejected. Defaults to automation; human gate is opt-in in practice.
- Positive human confirmation required per order/batch: Selected.

TRADEOFFS: Human gate adds latency to execution. For swing trading cadence (hold period hours to days), this latency is operationally acceptable. The gate creates a single point of human failure (operator unavailable). This requires documented operator availability SLA and escalation path.

VALIDATION_REQUIREMENTS:

- Verify gate is structural: automated test confirms no order reaches the broker API without a gate event

- Test gate under prompt injection scenario: verify AI component cannot bypass gate via crafted output
- Multi-sig override path: verify two distinct authorization identities are required

RISKS:

- Insider intentional bypass: documented failure mode. Mitigated by multi-sig for overrides and immutable audit trail.
- Operator unavailability: requires on-call SLA.

FUTURE_UPGRADE_TRIGGER: SEBI regulatory change permitting supervised autonomy. System scope change to execution strategies where human gate latency is disqualifying (intraday, HFT).

ARCHITECTURE_IMPACT: CRITICAL. Foundational execution architecture. Dependency for ARCH-011, ARCH-012.

ARCH-011 — AI Domain Boundary Architecture

ADR_ID: ARCH-011 **TITLE:** AI and Language Model Components Are Confined to the Research and Cognitive Domain — They Have No Pathway to Execution, Storage, or Order Routing **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** AI Boundary / Agent Boundaries

DECISION: AI and language model components operate exclusively within a Research/Cognitive domain. This domain has read access to validated, historical data for analysis. It has no write access to the validated data store. It has no access to order routing interfaces. It has no access to execution APIs. Its outputs are advisory signals consumed by human operators — they are not consumed directly by execution logic. Deterministic mathematical computations required for order sizing, risk calculation, and position management must be performed by deterministic code, not by language model inference. AI models must not perform chronological sorting, binary arithmetic, or any exact-precision numerical computation that feeds execution.

RATIONALE: Language models produce non-deterministic outputs. Binary math errors from LLMs (§ADR-099) and dynamic SQL hallucinations (§ADR-021, §ADR-101) are documented, catastrophic failure modes. The corpus rates direct AI execution safety at 10/100. Isolation to research domain is the architectural mechanism that makes this boundary enforceable and auditable.

SOURCE_CONSTRAINTS: ACD-AIB-001, ACD-AIB-002, ACD-AIB-003, ACD-AIB-004, ACD-AIB-005 **SOURCE_ADRS:** ADR-074, ADR-078, ADR-093, ADR-094, ADR-095, ADR-099, ADR-100, ADR-101

ALTERNATIVES_CONSIDERED:

- AI with execution access, policy-restricted: Rejected. Policy restriction is not architectural enforcement.
- AI in advisory role with outputs piped to execution: Rejected. Creates indirect execution path.
- AI isolated to read-only research domain, outputs consumed by human only: Selected.

TRADEOFFS: AI capabilities (pattern recognition, document analysis, research synthesis) are valuable. This architecture captures that value in the research domain while preventing the failure modes of AI-in-execution. The tradeoff is that AI cannot accelerate the execution path — only the research path.

VALIDATION_REQUIREMENTS:

- Architectural test: verify no code path from AI output buffer reaches order submission interface
- Verify AI-generated SQL is blocked from reaching the analytical data store directly
- Test AI output under adversarial input: verify output cannot trigger execution

RISKS:

- Domain boundary erosion over time: requires regular architectural audit
- AI sentiment-to-position-sizing math boundary unclear (ADR-098): the conversion math must be in deterministic code, not in AI inference

FUTURE_UPGRADE_TRIGGER: Emergence of AI execution paradigms with formal safety guarantees acceptable under SEBI. Regulatory framework for supervised algorithmic AI execution.

ARCHITECTURE_IMPACT: CRITICAL. Determines the fundamental trust topology of the system.

ARCH-012 — AI-to-Execution Physical Block Architecture

ADR_ID: ARCH-012 **TITLE:** The Interface Layer Between AI Components and System Tools

Must Physically Block Execution-Bound Tool Access — Policy Prohibition Is Insufficient
STATUS: APPROVED **ARCHITECTURE_DOMAIN:** AI Boundary / Security

DECISION: Any interface that connects an AI or language model component to system tools must implement physical, architectural blocking of execution-bound tool calls. The AI component must be incapable of calling execution tools — not merely policy-prohibited from doing so. This means execution tools must not exist in the tool registry visible to the AI component. Prompt injection must be treated as a first-class attack surface on this boundary. The tool interface must be designed such that a successfully injected adversarial prompt cannot reach execution-capable tools regardless of the injected instruction content.

RATIONALE: Policy-level prohibition without architectural enforcement is bypassed by prompt injection (§ADR-104). The boundary between the AI domain and execution domain must be physically enforced at the tool availability level — if the tool is not reachable, no injected instruction can invoke it. This is a distinct architecture decision from ARCH-011 (domain isolation) because it specifically addresses the attack surface of the AI tool interface.

SOURCE_CONSTRAINTS: ACD-AIB-006, ACD-SEC-001 **SOURCE_ADRS:** ADR-068, ADR-090, ADR-093, ADR-104

ALTERNATIVES_CONSIDERED:

- Policy-only prohibition: Rejected. Bypassable via prompt injection.
- Tool-level permission system: Rejected. Permissions can be manipulated via prompt injection if the permission check is in the AI's reasoning path.
- Physical exclusion of execution tools from AI tool registry: Selected.

TRADEOFFS: Reduces AI component flexibility. Requires maintaining two distinct tool registries (AI-accessible, execution-accessible). Adds architecture maintenance overhead. All tradeoffs accepted — the security surface of an AI with execution access is unacceptable.

VALIDATION_REQUIREMENTS:

- Adversarial prompt injection test: verify no crafted prompt reaches execution tools
- Verify tool registry separation is structural (code-enforced), not configurable
- Test on varied injection vectors: data feed content, document content, crafted user input

RISKS:

- Novel injection vectors not covered by test suite: requires ongoing security review
- GAP-07: FastMCP ASGI physical block empirical validation pending

FUTURE_UPGRADE_TRIGGER: Development of formally verified prompt injection defenses that provide equivalent protection without structural tool exclusion.

ARCHITECTURE_IMPACT: CRITICAL (security). Determines the security architecture of the AI interface boundary.

ARCH-013 — Risk Controls Architecture

ADR_ID: ARCH-013 **TITLE:** The System Must Implement Three Structural Risk Control Layers: Pre-Execution Sizing Controls, In-Execution Circuit Breakers, and Hard Configuration Caps **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Risk Controls

DECISION: Risk controls must be implemented as three structurally independent layers, each enforcing a different failure mode class:

Layer 1 — Pre-Execution Sizing Controls: Position size must be automatically reduced when model uncertainty exceeds a threshold. Parent-order balance checks must be hard-coded into execution loops — not asynchronous or optional. Order volume must be bounded as a percentage of recent market volume.

Layer 2 — In-Execution Circuit Breakers: Execution must automatically disconnect when consolidated tape latency exceeds a threshold. Market orders must be halted when order book depth falls below a minimum. Execution must halt when data feed delivers zero-volume candles that may indicate dropped packets rather than genuine zero-volume.

Layer 3 — Hard Configuration Caps: Bespoke scenario limits must be capped at values that cannot be overridden by individual operators. Variation margin release must be restricted during elevated volatility. These caps must be encoded in configuration that requires multi-party authorization to change (see ARCH-010).

RATIONALE: The three-layer structure ensures that each layer catches what the others miss. Layer 1 prevents oversized bets before they enter the market. Layer 2 responds to live market conditions during execution. Layer 3 prevents governance erosion over time. No single layer is sufficient alone.

SOURCE_CONSTRAINTS: ACD-VAL-007, ACD-REL-001, ACD-REL-002, ACD-REL-003, ACD-REL-004, ACD-REL-006 **SOURCE_ADRS:** ADR-032, ADR-035, ADR-036, ADR-037, ADR-038, ADR-049, ADR-050, ADR-051, ADR-052, ADR-054, ADR-055, ADR-057, ADR-060, ADR-089, ADR-113

ALTERNATIVES_CONSIDERED:

- Single risk limit layer: Rejected. Single point of failure.
- Alert-only risk monitoring: Rejected. Requires human response time during fast market events.
- Three structurally independent enforcement layers: Selected.

TRADEOFFS: Multiple independent layers add complexity and potential for conflicting signals (e.g., Layer 1 sizes down while Layer 2 triggers a halt simultaneously). Requires clear precedence logic. Accepted — redundant safety is the correct tradeoff for capital-at-risk systems.

VALIDATION_REQUIREMENTS:

- Replay March 2020 scenario through all three layers; verify correct response at each layer
- Test parent-order balance check: verify it fires synchronously, blocking child order submission
- Test circuit breaker: verify execution disconnects on latency breach, not on next cycle
- Test hard caps: verify individual operator cannot override without multi-party authorization

RISKS:

- Layer interactions: undefined behavior when multiple layers fire simultaneously requires documented precedence
- False positive circuit breaker during temporary latency spike: threshold calibration required

FUTURE_UPGRADE_TRIGGER: New risk failure modes identified via post-mortem analysis. Regulatory change requiring additional risk control categories.

ARCHITECTURE_IMPACT: HIGH. Determines execution safety architecture and the interface between signal computation and order submission.

ARCH-014 — Production Binary and Configuration Integrity Architecture

ADR_ID: ARCH-014 **TITLE:** All Production Nodes Must Run Identical, Hash-Verified Binaries — No Partial Rollouts, No Deprecated Code, No Reactivatable Configuration Flags **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Auditability / Operational

DECISION: Before any node is permitted to route live orders, it must pass three integrity checks: (1) binary hash verification against a known-good reference confirming it runs the correct version; (2) configuration flag audit confirming no deprecated namespace is present or reactivatable; (3) confirmation that all nodes in the cluster are running the identical binary simultaneously. Deprecated code must be permanently purged from the deployable codebase — commented-out, conditionally disabled, or flag-gated deprecated code does not satisfy the purge requirement. Deployment must be coordinated across all nodes atomically — partial rollout where nodes run different versions simultaneously is prohibited.

RATIONALE: Knight Capital lost $440M in 45 minutes from a deprecated code flag being reactivated on one node during a partial rollout. The corpus identifies this as its highest-priority operational failure mode (§7 Finding 1, §11 Req 2). Binary hash verification + configuration flag audit + atomic deployment are the three structural controls that prevent this failure class.

SOURCE_CONSTRAINTS: ACD-VAL-003, ACD-VAL-004, ACD-GOV-001, ACD-GOV-003
SOURCE_ADRS: ADR-009, ADR-030, ADR-031, ADR-054, ADR-081, ADR-082, ADR-083, ADR-108

ALTERNATIVES_CONSIDERED:

- Manual deployment review: Rejected. Human error rate unacceptably high for this failure class.
- Hash verification without config audit: Rejected. Correct binary can contain reactivatable deprecated config.
- Hash verification + config audit + atomic deployment: Selected.

TRADEOFFS: Atomic deployment increases deployment complexity and rollback difficulty. Accepted — the alternative (partial deployment) has a documented catastrophic failure mode.

VALIDATION_REQUIREMENTS:

- Verify hash verification fires before any order routing begins
- Test config flag audit: inject deprecated flag namespace; verify audit catches it before deployment completes
- Test partial rollout attempt: verify it is blocked
- Verify deprecated code purge: confirm no path to legacy execution mode exists in codebase

RISKS:

- Automated audit has incomplete flag namespace registry: requires maintained registry of all deprecated namespaces

- Hash verification spoofed: requires trusted reference hash storage

FUTURE_UPGRADE_TRIGGER: Change in deployment architecture (e.g., containerization) that provides equivalent guarantees via different mechanism.

ARCHITECTURE_IMPACT: HIGH (operational safety). Prevents Knight Capital failure class.

ARCH-015 — Survivorship Bias Elimination in Backtesting Data

ADR_ID: ARCH-015 **TITLE:** Backtesting Datasets Must Include Delisted Instruments for the Full Historical Period — Survivorship-Biased Datasets Are Prohibited as Sole Backtesting Input **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Backtesting Integrity / Data Governance

DECISION: The backtesting data infrastructure must maintain and include delisted instruments in the historical universe for the relevant backtesting period. Any dataset that excludes stocks that were delisted during the backtesting period is survivorship-biased and must not serve as the sole input to strategy validation. The data pipeline must track and ingest delisting events as first-class data. The validation pipeline must enforce that the backtesting universe is survivorship-corrected before a strategy is permitted to proceed to the deployment gate.

RATIONALE: Survivorship bias systematically inflates backtest performance by removing the worst outcomes (companies that failed) from the test universe. Strategies validated on survivorship-biased data produce optimistic metrics that do not reflect live market performance.

SOURCE_CONSTRAINTS: ACD-DAT-003 **SOURCE_ADRS:** ADR-013

ALTERNATIVES_CONSIDERED:

- Backtest on current surviving universe only: Rejected. Survivorship bias.
- Survivorship-corrected universe as mandatory input: Selected.

TRADEOFFS: Survivorship-corrected data is harder to source and maintain. Delisted instrument data may be incomplete from some providers. Accepted — the alternative produces invalid strategy validation metrics.

VALIDATION_REQUIREMENTS:

- Verify backtesting engine rejects a survivorship-biased dataset (no delisted instruments present)
- Confirm delisting events are tracked as first-class data in the ingestion pipeline
- Cross-check: universe at historical date T must include all instruments active at T, including subsequently delisted ones

RISKS:

- Survivorship-corrected data not available from all providers: may require premium data sources
- Partial delist coverage: some delisted instruments may have incomplete historical records — handling policy required

FUTURE_UPGRADE_TRIGGER: Change in regulatory requirements for backtesting data quality. Addition of new markets with different delisting data availability.

ARCHITECTURE_IMPACT: MEDIUM. Determines backtesting data architecture and pipeline requirements.

ARCH-016 — API Rate Compliance and Retry Architecture

ADR_ID: ARCH-016 **TITLE:** All Broker API Interactions Must Comply With Rate Limits via Architectural Enforcement — Retry Logic Must Be Deterministic, Bounded, and Exponential **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** API Rate Compliance / Regulatory

DECISION: The execution pathway must implement architectural rate limiting that enforces SEBI-mandated order submission limits (10 orders/second) as a hard ceiling. The rate limiter must be in the critical path — it cannot be a best-effort or advisory component. All retry logic on API rate-limit responses must be: (1) deterministic (same input → same retry schedule); (2) bounded (maximum retry attempt ceiling defined and enforced); (3) exponential (interval grows between attempts). Immediate re-retry after a 429 response is prohibited. Shadow IP ban behavior from brokers (permanent block without HTTP error) must be treated as a distinct failure mode from rate limiting and must be detectable.

RATIONALE: SEBI mandates 10 orders/second. Exceeding this rate risks account suspension (§7 Finding 14). Unbounded retry storms after 429 responses will exhaust rate budgets and trigger shadow bans (§ADR-085). These are architectural failures, not operational ones — they must be solved in the architecture.

SOURCE_CONSTRAINTS: ACD-REG-001, ACD-REL-005 **SOURCE_ADRS:** ADR-008, ADR-042, ADR-085, ADR-086, ADR-106

ALTERNATIVES_CONSIDERED:

- Application-level rate limiting (advisory): Rejected. Bypassable under high load.
- No retry logic: Rejected. Transient failures cause unnecessary execution gaps.

- Architectural rate limit enforcement + deterministic bounded exponential retry: Selected.

TRADEOFFS: Architectural rate limiter adds a component to the execution path. Bounded retry means some transient failures are not retried to success. Accepted — the alternative failure modes (account suspension, shadow ban) are unacceptable.

VALIDATION_REQUIREMENTS:

- Sandbox test: verify rate limiter blocks order submission above 10/second ceiling
- Test: inject HTTP 429 response; verify retry fires with exponential backoff and stops at ceiling
- Test: distinguish shadow ban from rate limit response; verify different handling paths

RISKS:

- Shadow ban vs. 429 disambiguation may not be possible via API response alone: requires behavioral detection heuristic
- Rate limit changes by SEBI or broker: architectural limiter must be reconfigurable

FUTURE_UPGRADE_TRIGGER: SEBI regulatory change to rate limit parameters. Broker API change to rate limit enforcement mechanism.

ARCHITECTURE_IMPACT: MEDIUM-HIGH. Determines execution pathway architecture and API interface design.

ARCH-017 — Secrets and Authentication Architecture

ADR_ID: ARCH-017 **TITLE:** All Credentials and Authentication Tokens Must Be Managed Outside the Application Runtime — Token Lifecycle Must Be Automated **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Security / Authentication

DECISION: API keys, OAuth credentials, and session tokens must never be embedded in source code, configuration files committed to version control, or environment variables accessible without access control. Credentials must be managed through a controlled secrets management mechanism with access logging. Authentication token lifecycle (issuance, refresh, expiry) must be fully automated. Any authentication flow that requires daily manual human intervention (including 2FA) to maintain system connectivity is incompatible with unattended operation and must be resolved at the architecture level before production deployment.

RATIONALE: Credential exposure through code repositories is a primary security failure mode. OAuth tokens with manual daily refresh will halt the trading system during unattended hours (§9 A22). SEBI OAuth mandate (§7 Finding 14) must be satisfied by automated token management.

SOURCE_CONSTRAINTS: ACD-SEC-002, ACD-OPS-002 **SOURCE_ADRS:** ADR-008, ADR-087, ADR-117

ALTERNATIVES_CONSIDERED:

- Credentials in config files: Rejected. Version control exposure risk.
- Manual daily token refresh: Rejected. Incompatible with unattended operation.
- Automated token lifecycle + external secrets management: Selected.

TRADEOFFS: Automated token refresh requires solving the 2FA problem for brokers that mandate it. This may require broker API negotiation or a specific technical solution (e.g., broker-provided programmatic refresh). This is an unresolved validation requirement before production.

VALIDATION_REQUIREMENTS:

- Verify no credentials present in source code or committed configuration
- Test automated token refresh without manual intervention over 48-hour window
- Verify secrets management access log is maintained

RISKS:

- Broker OAuth 2FA cannot be automated: open validation blocker (ADR-117). Must be resolved before production deployment.

FUTURE_UPGRADE_TRIGGER: Broker changes to authentication mechanism. Regulatory change to authentication requirements.

ARCHITECTURE_IMPACT: MEDIUM. Security baseline for all external connectivity.

ARCH-018 — Corporate Actions Data Adjustment Requirement

ADR_ID: ARCH-018 **TITLE:** All Price Data Used in Signal Computation Must Be Adjusted for Corporate Actions — Unadjusted Data Is Prohibited in Signal Pipelines **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Data Governance / Market Data Reliability

DECISION: The data pipeline must apply corporate action adjustments (splits, dividends, mergers, demergers) to all historical and live price data before it is written to the validated data store. Unadjusted data must not enter the signal computation pipeline. Adjustment methodology must be documented and versioned. Re-adjustment events (retroactive corporate action corrections by providers) must be handled as explicit pipeline events that trigger downstream recomputation, not silent overwrites.

RATIONALE: Unadjusted price data produces artificial price discontinuities at corporate action events. Split-unadjusted data shows a false price drop that generates false signals. Yahoo Finance adjusted close mis-adjustment is a documented provider-specific risk (§ADR-010). The adjustment pipeline is a data integrity requirement, not a data enrichment concern.

SOURCE_CONSTRAINTS: ACD-REG-004 **SOURCE_ADRS:** ADR-005, ADR-006, ADR-010

ALTERNATIVES_CONSIDERED:

- Adjust at signal computation time: Rejected. Adjustment math runs per-signal, introducing computation overhead and inconsistency risk.
- Adjust at storage time with versioned adjustment log: Selected.

TRADEOFFS: Versioned adjustment log adds storage and pipeline complexity. Retroactive re-adjustment events require reprocessing of derived signals. Accepted.

VALIDATION_REQUIREMENTS:

- Inject a synthetic split event; verify adjusted price series is correct before and after split date
- Test retroactive re-adjustment: verify downstream recomputation triggers correctly
- Verify adjustment methodology matches provider documentation

RISKS:

- Provider adjustment methodology differs from system adjustment methodology: requires explicit cross-validation
- Demerger adjustment math: complex, requires explicit case handling

FUTURE_UPGRADE_TRIGGER: Regulatory change to adjustment methodology standards. New corporate action types requiring new adjustment logic.

ARCHITECTURE_IMPACT: MEDIUM. Data pipeline correctness for all historical and live data.

ARCH-019 — Scalability Transition Architecture

ADR_ID: ARCH-019 **TITLE:** The Architecture Must Define a Validated Scalability Boundary — Horizontal Scaling Requires a Deliberate Architecture Transition, Not an Incremental Expansion **STATUS:** APPROVED **ARCHITECTURE_DOMAIN:** Scalability

DECISION: The current local-first embedded architecture (ARCH-002) defines a scalability ceiling. The architecture must define the validated boundary at which that ceiling is reached (data volume, operator count, concurrent query load) and must treat crossing that boundary as a deliberate architecture transition, not an organic scaling decision. The transition from local-first embedded to a distributed architecture is a first-class architecture event that requires a new ADR set. No incremental addition of distributed components to the current architecture is permitted without triggering this transition review. The scalability boundary must be empirically validated (benchmark tested), not assumed.

RATIONALE: The corpus identifies DuckDB/SQLite concurrency limits at 50GB scale as a critical open gap (§22, GAP-08). Cloud deployment topology Stage 2→4 is entirely unresearched (GAP-12). These are not operational parameters — they are architecture transition triggers. Treating them as incremental decisions risks building past the architecture's validated boundary.

SOURCE_CONSTRAINTS: ACD-DAT-005 **SOURCE_ADRS:** ADR-023, ADR-084, ADR-092, ADR-119

ALTERNATIVES_CONSIDERED:

- Organic scaling (add components as needed): Rejected. Creates hybrid architecture with undefined failure modes at the local/distributed boundary.
- Define transition boundary upfront, validate empirically, treat crossing as a deliberate event: Selected.

TRADEOFFS: Requires upfront benchmark work to define the boundary. Prevents convenient "just add a server" decisions. Accepted — undefined architecture boundaries are a systemic risk.

VALIDATION_REQUIREMENTS:

- Benchmark: determine actual concurrency ceiling of local embedded architecture at production data scale
- Benchmark: determine maximum data volume before query performance degrades below acceptable threshold
- Define and document the scalability boundary as a versioned architecture parameter

RISKS:

- Boundary not established before production: system deployed without knowing its own capacity limits
- Benchmark conditions not representative of production load: requires realistic test data

FUTURE_UPGRADE_TRIGGER: Empirical validation shows current architecture approaching its scalability boundary. Operator count growth. Asset universe expansion. Strategy count expansion.

ARCHITECTURE_IMPACT: MEDIUM (now), HIGH (when transition triggered). Defines the architecture's own lifecycle.

ARCH-020 — Concurrency Boundary Between Analytical and Transactional Storage

ADR_ID: ARCH-020 **TITLE:** The Concurrency Boundary Between Analytical Query Engine and Transactional Record Store Must Be Empirically Validated Before Architecture Is Finalized **STATUS:** APPROVED — VALIDATION PENDING **ARCHITECTURE_DOMAIN:** Concurrency Boundary / Storage Strategy

DECISION: When an analytical query engine and a transactional record store are co-embedded in the same process (per ARCH-002), their concurrent access patterns must be empirically benchmarked at production data scales before the architecture is considered finalized. Specifically: analytical long-running scan queries must not block transactional writes that are time-sensitive for order execution. The concurrency architecture must define whether these two access patterns share a scheduler, compete for the same memory pool, or are isolated. Isolation strategy must be explicit — not assumed safe by default. This ADR is approved as an architecture requirement; it transitions to fully resolved status when the benchmark validation is complete.

RATIONALE: DuckDB/SQLite concurrent access benchmarks at 50GB+ scale are absent from the corpus (GAP-08). An analytical query blocking a transactional write during live order execution is a latency failure mode with direct capital impact. This is not a performance optimization — it is an architecture correctness requirement.

SOURCE_CONSTRAINTS: ACD-DAT-005, ACD-DAT-006 **SOURCE_ADRS:** ADR-023, ADR-047, ADR-088, ADR-119

ALTERNATIVES_CONSIDERED:

- Assume concurrent access is safe (no benchmark): Rejected. Undocumented assumption with critical failure mode.

- Separate processes for analytical and transactional (IPC): Alternative — acceptable if benchmark shows shared-process concurrency is unsafe.
- Shared-process co-embedding with explicit isolation mechanism: Selected as initial target pending benchmark.

TRADEOFFS: Shared process is simpler (no IPC) but may introduce contention. Separate processes add IPC overhead and deployment complexity but provide guaranteed isolation. The correct tradeoff cannot be determined without the benchmark.

VALIDATION_REQUIREMENTS:

- Benchmark: concurrent analytical scan (multi-year 1-minute OHLCV) + transactional write under live execution simulation
- Measure: write latency during active long-running scan
- Define: maximum acceptable write latency for execution-critical transactions
- Decision: if benchmark exceeds threshold, transition to process-separated architecture

RISKS:

- Architecture is finalized without benchmark completion: open blocker
- Benchmark conditions differ from production (dataset size, query patterns): requires production-representative test

FUTURE_UPGRADE_TRIGGER: Benchmark completion determines whether this ADR resolves as shared-process or triggers process-separation architecture decision.

ARCHITECTURE_IMPACT: HIGH (unresolved). This is the primary remaining technical architecture blocker.

SECTION 3 — ADR DEPENDENCY MAP

```
ARCH-001 (System Scope)
├── ARCH-010 (Human Approval Gate)
│   ├── ARCH-011 (AI Domain Boundary)
│   │   └── ARCH-012 (AI-to-Execution Physical Block)
│   └── ARCH-013 (Risk Controls)
│       └── ARCH-009 (Drift Detection)
└── ARCH-016 (API Rate Compliance)
    └── ARCH-017 (Secrets & Auth)
```

```
ARCH-002 (Local-First Storage)
├── ARCH-003 (Market Data Format)
│   └── ARCH-006 (Data Ingestion Integrity)
│       ├── ARCH-004 (Multi-Source Verification)
│       │   └── ARCH-018 (Corporate Actions)
│       └── ARCH-015 (Survivorship Bias)
├── ARCH-005 (Transaction Integrity)
└── ARCH-020 (Concurrency Boundary) → [VALIDATION PENDING]
    └── ARCH-019 (Scalability Transition)

ARCH-007 (Walk-Forward Gate)
├── ARCH-008 (Statistical Threshold)
└── ARCH-014 (Binary & Config Integrity)
```

Critical Path: ARCH-001 → ARCH-010 → ARCH-011 → ARCH-012 **Data Path:** ARCH-002 → ARCH-003 → ARCH-004 → ARCH-006 **Validation Path:** ARCH-007 → ARCH-008 → ARCH-009 **Open Blocker:** ARCH-020 must resolve before ARCH-002 is fully finalized

---

SECTION 4 — ARCHITECTURE DECISION HIERARCHY

```
TIER 1 — FOUNDATIONAL (Cannot change without rebuilding from scratch)
  ARCH-001  System Scope: SEBI-Regulated Deterministic Human-Gated System
  ARCH-002  Local-First Embedded Storage Architecture
  ARCH-010  Human Approval Gate Architecture
  ARCH-011  AI Domain Boundary Architecture

TIER 2 — STRUCTURAL (Change requires significant redesign)
  ARCH-003  Market Data Storage Format
  ARCH-004  Market Data Multi-Source Verification
  ARCH-005  Transaction Integrity & Crash Recovery
  ARCH-007  Walk-Forward Deployment Gate
  ARCH-012  AI-to-Execution Physical Block
  ARCH-013  Risk Controls Three-Layer Architecture
  ARCH-014  Production Binary & Config Integrity

TIER 3 — COMPONENT (Change requires component redesign, not system redesign)
  ARCH-006  Data Ingestion Integrity Layer
  ARCH-008  Statistical Deployment Threshold
```

```
  ARCH-009  Drift Detection & Regime-Aware Execution
  ARCH-015  Survivorship Bias Elimination
  ARCH-016  API Rate Compliance & Retry Architecture
  ARCH-017  Secrets & Authentication Architecture
  ARCH-018  Corporate Actions Adjustment
  ARCH-019  Scalability Transition Boundary

TIER 4 — VALIDATION GATE (Must resolve before Tier 2 parent is finalized)
  ARCH-020  Concurrency Boundary [VALIDATION PENDING]
```

SECTION 5 — REJECTED OR DERIVED REGISTRY

The following 100 ADRs were excluded from the Architecture ADR Corpus. They are retained here as a traceable record. They are not lost — they become implementation requirements, operational procedures, and validation test cases derived from the 20 Architecture ADRs above.

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-002	SQLite Exclusion from Standalone Time-Series	Derived — expresses the same decision as ARCH-002 at implementation detail level	ARCH-002
ADR-005	Corporate Actions Mandatory Split-Adjusted	Merged — absorbed into ARCH-018	ARCH-018
ADR-006	Upstox Uplink Provider Selection	Technology Specific — names a provider; constraint expressed in ARCH-004	ARCH-004
ADR-009	Production Binary Hygiene	Operational — procedure derived from ARCH-014	ARCH-014
ADR-010	Yahoo Finance Mis-Adjustment Risk	Technology Specific + Weak Evidence	ARCH-018
ADR-013	Survivorship Bias Delisted Stock	Merged — absorbed into ARCH-015	ARCH-015
ADR-014	NLP Domain Validity US Models	Weak Evidence — insufficient to drive architecture	ARCH-011
ADR-015	SQLite WAL Litestream S3 Recovery	Derived — implementation mechanism for ARCH-005	ARCH-005
ADR-016	Synthetic Anomaly Injection Testing	Validation Requirement — derived from ARCH-006	ARCH-006
ADR-017	Bad Pricing Infinite Loop Failure Mode	Derived — failure mode documented in ARCH-006	ARCH-006
ADR-018	WebSocket Latency Spike Risk	Derived — failure mode documented in ARCH-013	ARCH-013
ADR-019	Exchange Trade Erasure Handling	Merged — absorbed into ARCH-005	ARCH-005
ADR-020	SQLite WAL NSE Trade Void Events	Derived — implementation detail of ARCH-005	ARCH-005
ADR-021	LLM Dynamic SQL Hallucination Risk	Derived — failure mode documented in ARCH-011	ARCH-011

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-022	LLM Context Window Degradation	Technology Specific	ARCH-011
ADR-023	DuckDB/SQLite Concurrency Gap	Merged — absorbed into ARCH-020	ARCH-020
ADR-024	Non-Ergodic VaR Missing Research	Research Gap — not an architecture decision	ARCH-013
ADR-025	Multi-Broker Aggregate Margin Gap	Research Gap	ARCH-013
ADR-026	Yahoo Finance 15-Min Delay	Technology Specific + Weak Evidence	ARCH-004
ADR-027	Walk-Forward CV over K-Fold	Merged — absorbed into ARCH-007	ARCH-007
ADR-028	Minimum t-stat > 3.0 Threshold	Merged — absorbed into ARCH-008	ARCH-008
ADR-029	Rejection of t=2.0 as Gate	Merged — absorbed into ARCH-008	ARCH-008
ADR-030	Cluster Binary Hash Verification	Merged — absorbed into ARCH-014	ARCH-014
ADR-031	Config Flag Deprecated Memory Audit	Merged — absorbed into ARCH-014	ARCH-014
ADR-032	Parent-Order Balance Checks	Merged — absorbed into ARCH-013 Layer 1	ARCH-013
ADR-033	Chaos Tests Before Production Routing	Validation Requirement	ARCH-014
ADR-034	PSI Tracking Concept Drift	Merged — absorbed into ARCH-009	ARCH-009
ADR-035	Bid Size Reduction Under Uncertainty	Merged — absorbed into ARCH-013 Layer 1	ARCH-013

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-036	Hard Volume % Execution Limit	Merged — absorbed into ARCH-013 Layer 1	ARCH-013
ADR-037	Consolidated Tape Latency Threshold	Merged — absorbed into ARCH-013 Layer 2	ARCH-013
ADR-038	Order Book Depth Monitoring	Merged — absorbed into ARCH-013 Layer 2	ARCH-013
ADR-039	90% Branch Coverage Backtesting	Validation Requirement	ARCH-007
ADR-040	Intraday Minute Feed Completeness	Merged — absorbed into ARCH-006	ARCH-006
ADR-041	Cross-Verify OHLCV Two Providers	Merged — absorbed into ARCH-004	ARCH-004
ADR-042	Sandbox Broker Rate Limit Test	Validation Requirement	ARCH-016
ADR-043	SQLite WAL S3 Recovery Hard Power-Off	Validation Requirement	ARCH-005
ADR-044	Synthetic Anomaly Parquet Injection	Validation Requirement	ARCH-006
ADR-045	FastMCP Prompt Injection Validation	Validation Requirement	ARCH-012
ADR-046	Walk-Forward OOS Regime Fidelity	Validation Requirement / Accepted Risk	ARCH-007
ADR-047	Cron Job Overlap Deadlock Risk	Merged — absorbed into ARCH-020	ARCH-020
ADR-048	Autoencoder Anomaly Classification	Validation Requirement	ARCH-009
ADR-049	Hard Position Limit API Disconnection	Merged — absorbed into ARCH-013 Layer 2	ARCH-013

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-050	Circuit Breaker Trend-Following Hedge	Merged — absorbed into ARCH-013 Layer 2	ARCH-013
ADR-051	Informational Cascade Volume Halt	Merged — absorbed into ARCH-013 Layer 2	ARCH-013
ADR-052	Dynamic Margin Auto-Scale Illiquidity	Merged — absorbed into ARCH-013 Layer 1	ARCH-013
ADR-053	Multi-Broker Concentration Controls	Merged — absorbed into ARCH-013 Layer 3	ARCH-013
ADR-054	Bespoke Scenario Limit Hard Caps	Merged — absorbed into ARCH-013 Layer 3	ARCH-013
ADR-055	Variation Margin Release Restriction	Merged — absorbed into ARCH-013 Layer 3	ARCH-013
ADR-056	Convergence Arbitrage Stress Testing	Validation Requirement	ARCH-013
ADR-057	ML Uncertainty Execution Scale-Down	Merged — absorbed into ARCH-013 Layer 1	ARCH-013
ADR-058	ML Concept Drift Re-Anchoring	Merged — absorbed into ARCH-009	ARCH-009
ADR-059	High-Velocity Operational Deployment Risk	Governance — expresses constraint, not architecture	ARCH-014
ADR-060	Data Feed Latency Circuit Breaker	Merged — absorbed into ARCH-013 Layer 2	ARCH-013
ADR-061	Non-Ergodic VaR Framework	Research Gap — no mathematical framework yet exists	ARCH-013
ADR-062	Slippage Transaction Cost Controls	Validation Requirement	ARCH-013
ADR-063	Denoising Autoencoder Masking Audit	Validation Requirement	ARCH-009

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-064	Risk Committee Oversight Mandate	Governance Procedure	ARCH-010
ADR-065	TRS Margin Audit Prime Brokerage	Operational	ARCH-013
ADR-066	Multi-Broker Collateral Fire Sale Prevention	Merged — absorbed into ARCH-013 Layer 3	ARCH-013
ADR-067	LLM Direct Trade Execution Prohibition	Merged — absorbed into ARCH-011	ARCH-011
ADR-068	Physical FastMCP Execution Boundary	Merged — absorbed into ARCH-012	ARCH-012
ADR-069	Multi-Sig Human Approval Overrides	Merged — absorbed into ARCH-010	ARCH-010
ADR-070	Human Gate for AI-Influenced Pricing	Merged — absorbed into ARCH-010	ARCH-010
ADR-072	Alert Fatigue Runbook Prevention	Operational Procedure	ARCH-010
ADR-073	Active Risk Committee Governance	Governance Procedure	ARCH-010
ADR-074	Autonomous Agent Math/Storage Prohibition	Merged — absorbed into ARCH-011	ARCH-011
ADR-075	Quantile Regression Uncertainty Band Review	Weak Evidence	ARCH-013
ADR-076	Autonomous Volume Growth Prohibition	Merged — absorbed into ARCH-010	ARCH-010
ADR-077	Embedded Zero-Copy Storage (DuckDB+SQLite)	Technology Specific — constraint expressed in ARCH-002	ARCH-002
ADR-078	Hive-Partitioned Parquet Mandatory Format	Technology Specific — constraint expressed in ARCH-003	ARCH-003

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-079	SQLite WAL Management S3 Replication	Implementation Detail	ARCH-005
ADR-080	Async Backup S3 Litestream Recovery	Implementation Detail	ARCH-005
ADR-081	Automated Binary Hash Verification	Merged — absorbed into ARCH-014	ARCH-014
ADR-082	Config Flag Namespace Audit	Merged — absorbed into ARCH-014	ARCH-014
ADR-083	Coordinated Deployment Strategy	Merged — absorbed into ARCH-014	ARCH-014
ADR-084	Cloud Deployment Stage 2→4 Transition	Merged — absorbed into ARCH-019	ARCH-019
ADR-085	Deterministic Exponential Backoff	Merged — absorbed into ARCH-016	ARCH-016
ADR-086	SEBI API Rate Limit Static IP OAuth	Merged — absorbed into ARCH-001, ARCH-016	ARCH-001, ARCH-016
ADR-087	OAuth Token Auto-Refresh	Merged — absorbed into ARCH-017	ARCH-017
ADR-088	Cron Job Overlap Deadlock Prevention	Merged — absorbed into ARCH-020	ARCH-020
ADR-089	Execution Circuit Breaker Tape Latency	Merged — absorbed into ARCH-013 Layer 2	ARCH-013
ADR-090	FastMCP ASGI Co-hosting	Technology Specific + Deferred	ARCH-012
ADR-091	WebSocket Packet Loss Volatility	Derived failure mode	ARCH-006, ARCH-013
ADR-092	DuckDB Parquet Scan Memory Limits	Merged — absorbed into ARCH-020	ARCH-020

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-093	Strict LLM Execution Prohibition FastMCP	Merged — absorbed into ARCH-011	ARCH-011
ADR-094	AI Domain Segregation Cognitive vs Execution	Merged — absorbed into ARCH-011	ARCH-011
ADR-095	Prohibition Fully Autonomous AI Grids	Merged — absorbed into ARCH-001, ARCH-011	ARCH-001, ARCH-011
ADR-096	Human-in-the-Loop AI Pricing Gate	Merged — absorbed into ARCH-010	ARCH-010
ADR-097	Model Uncertainty Execution Halt Trigger	Merged — absorbed into ARCH-013 Layer 1	ARCH-013
ADR-098	AI Sentiment to Kelly Fraction Math	Merged — absorbed into ARCH-011	ARCH-011
ADR-099	LLM Prohibition Deterministic Sort/Binary Math	Merged — absorbed into ARCH-011	ARCH-011
ADR-100	Auto-Coder AI Backtester Prohibition	Merged — absorbed into ARCH-011	ARCH-011
ADR-101	LLM Dynamic SQL Against DuckDB Prohibited	Merged — absorbed into ARCH-011	ARCH-011
ADR-102	LLM Model Routing Tier Boundary	Technology Specific	ARCH-011
ADR-103	Local SLM Sentiment Tagging Capability	Technology Specific + Weak Evidence	ARCH-011
ADR-104	Prompt Injection Defense FastMCP Physical Block	Merged — absorbed into ARCH-012	ARCH-012
ADR-105	Automated Binary Hash Before Production	Derived — implementation of ARCH-014	ARCH-014
ADR-106	Deterministic Exponential Backoff HTTP 429	Derived — implementation of ARCH-016	ARCH-016

ADR_ID	Title (abbreviated)	Reason for Exclusion	Derived From
ADR-107	Parent-Order Balance Checks Execution Loops	Derived — implementation detail of ARCH-013	ARCH-013
ADR-108	Deprecated Code Purge Config Flag	Derived — implementation of ARCH-014	ARCH-014
ADR-109	Alert Fatigue Dedicated Runbooks	Operational Procedure	ARCH-010
ADR-110	Validated Configuration Rollback Procedure	Operational Procedure	ARCH-014
ADR-111	Automated Chaos Sanity Tests	Validation Requirement	ARCH-014
ADR-112	Execution Algorithm Disconnection Tape Latency	Derived — implementation of ARCH-013 Layer 2	ARCH-013
ADR-113	Buy-Side Order Book Depth Monitoring	Derived — implementation of ARCH-013 Layer 2	ARCH-013
ADR-114	Capital Buffer Retroactive Exchange Cancellation	Merged — absorbed into ARCH-005	ARCH-005
ADR-115	SQLite WAL S3 Litestream Reliability	Validation Requirement	ARCH-005
ADR-116	Cron Job Overlap Deadlock Prevention	Derived — implementation of ARCH-020	ARCH-020
ADR-117	OAuth Token Refresh Without 2FA	Validation Requirement (OPEN BLOCKER)	ARCH-017
ADR-118	Incorrect Pricing Circuit Breaker	Merged — absorbed into ARCH-006, ARCH-013	ARCH-006, ARCH-013
ADR-119	DuckDB SQLite Concurrency Benchmark	Merged — absorbed into ARCH-020	ARCH-020
ADR-120	API Stability Uptime Extreme Events	Research Gap	ARCH-004

SECTION 6 — ARCHITECTURE READINESS ASSESSMENT

Open Blockers Inherited from Corpus

The following validation gaps must be resolved before architecture design is finalized. These are not future concerns — they are open questions whose answers change architectural decisions.

Blocker	Affects	Status	Required Action
GAP-08: DuckDB/SQLite concurrency at 50GB scale	ARCH-002, ARCH-020	UNRESOLVED	Benchmark: concurrent analytical scan + transactional write under production data load
ADR-117: OAuth 2FA automation with broker	ARCH-017	UNRESOLVED	Sandbox test: automated token refresh without manual 2FA over 48h window
ADR-015/043: WAL recovery under hard power-off	ARCH-005	UNRESOLVED	Chaos test: force power-off during write, verify recovery
ADR-019: Clearinghouse cancellation handling policy	ARCH-005	UNRESOLVED	Broker API documentation review + sandbox test
ADR-006/120: Broker API uptime during extreme events	ARCH-004, ARCH-016	UNRESOLVED	Empirical latency logging or broker SLA documentation

ADR Status Summary

Architecture ADR	Status	Validation Gate
ARCH-001	APPROVED	SEBI compliance documentation + sandbox
ARCH-002	APPROVED	Benchmark pending (ARCH-020)
ARCH-003	APPROVED	Schema validation test
ARCH-004	APPROVED	Cross-verification pipeline test
ARCH-005	APPROVED — VALIDATION PENDING	WAL recovery chaos test

Architecture ADR	Status	Validation Gate
ARCH-006	APPROVED	Anomaly injection test
ARCH-007	APPROVED	Pipeline temporal ordering test
ARCH-008	APPROVED	Gate enforcement test
ARCH-009	APPROVED	March 2020 replay test
ARCH-010	APPROVED	Prompt injection gate test
ARCH-011	APPROVED	Domain boundary audit
ARCH-012	APPROVED — GAP-07 PENDING	Adversarial injection test
ARCH-013	APPROVED	Three-layer scenario replay
ARCH-014	APPROVED	Hash + config + deployment tests
ARCH-015	APPROVED	Survivorship bias pipeline test
ARCH-016	APPROVED	Rate limit + retry tests
ARCH-017	APPROVED — ADR-117 PENDING	48h OAuth automation test
ARCH-018	APPROVED	Split event injection test
ARCH-019	APPROVED	Benchmark defines boundary
ARCH-020	APPROVED — VALIDATION PENDING	Concurrency benchmark (BLOCKER)

SECTION 7 — FINAL VERDICT

Can Architecture Design Begin?

NO

Justification

The Architecture ADR Corpus is complete. The 20 ADRs above are internally consistent, non-

redundant, and together represent the full set of architecture-driving decisions. They are ready to serve as the single source of truth for architecture planning.

However, architecture design cannot begin because two blockers are unresolved that will change architectural decisions — not just validate them:

Blocker 1 — ARCH-020 (Concurrency Boundary): The benchmark determining whether the local embedded analytical engine and transactional store can safely coexist in the same process at production data scales has not been run. If the benchmark shows contention above the acceptable threshold, ARCH-002 must be revised to a process-separated architecture. This changes the fundamental storage topology — a core Tier 2 structural decision. Architecture cannot be drawn until this is known.

Blocker 2 — ARCH-017 / ADR-117 (OAuth Automation): Whether broker OAuth tokens can be refreshed automatically without daily manual 2FA has not been empirically validated. If they cannot, the authentication architecture requires a fundamentally different solution. Architecture that assumes automated token refresh and is built before this is validated will be built on an unproven foundation.

Conditions for YES

Architecture design may begin when:

1. ARCH-020 benchmark is complete and documents the concurrency ceiling.
2. ADR-117 OAuth automation is validated in broker sandbox.

Both conditions can be resolved in parallel. Neither requires the other to complete first. Estimated validation effort: 2–5 working days.

What Is Ready

Everything else is ready. The 20 Architecture ADRs are stable. The Tier 1 foundational decisions (scope, local-first, human gate, AI boundary) are fully validated by the existing corpus. Once the two remaining blockers are closed, architecture design can proceed immediately from this document without further ADR work.

Architecture ADR Synthesis Version 1 — Final This document supersedes: ADR Candidate Corpus, ADR Quality Audit, Constraint Registries Single source of truth for architecture planning No further ADR work required after blocker resolution